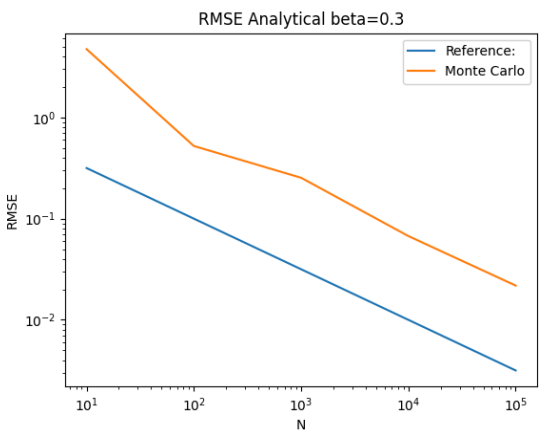
Leonid Scott

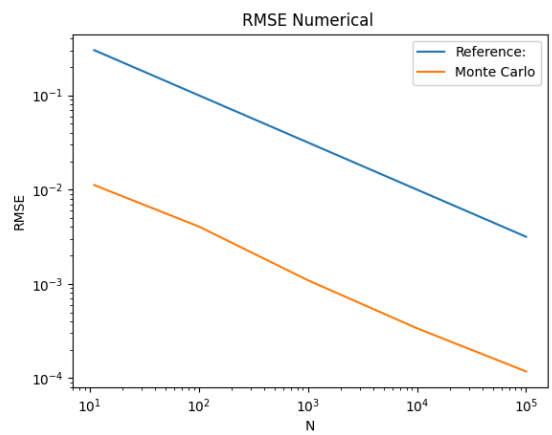
Uncertainty Quantification - Homework 1

Problem 1) Monte-Carlo and KDE

Part A) RMSE

The Root Mean Square Error follows the expected $O(N^{-1/2})$ trend. Simulations are run at N = [10,100,1000,10000,100000]. At each value of N, a batch of 50 monte-carlo simulations are run, and their standard deviation is computed to find RMSE at that particular value of N. Details on the statistical convergence properties can be found in the appendix. Details on the numerical discretization for the numerical model, and how the sampler works can be found in the appendix.

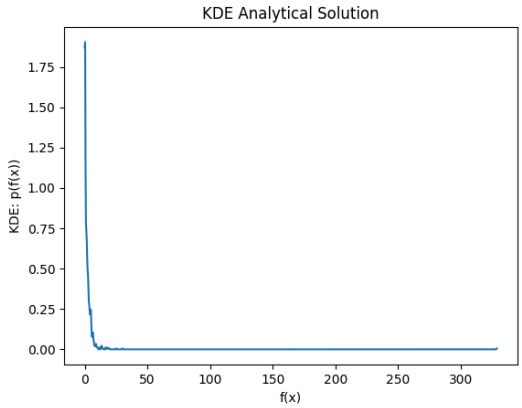


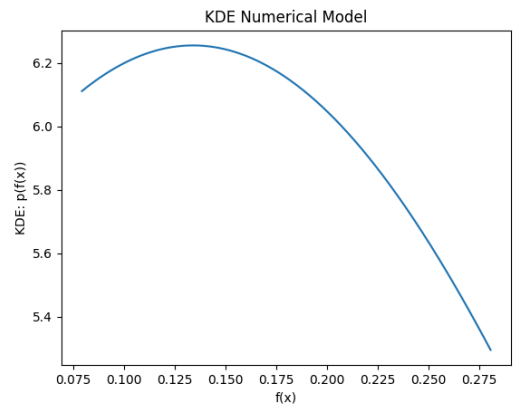



```python
T = TypeVar("T")
def monte_carlo(model: Callable[[T], float],
                sampler: Callable[[],T],
                N: int) -> Tuple[float, float]:
    samples = (pseq(range(N))
               .map(lambda _: model(sampler())).to_list())
    exp = 1.0/N * sum(samples)
    var = 1.0/(N-1.0) * (seq(samples)
                         .map(lambda s: (s-exp)**2)
                         .sum())
    return exp,var
```

Part B) KDE

Here, KDE was calculated with 1000 samples, with bandwidth tuning according to Scott's rule (naturally). Both KDE's seem to behave like log-normal distributions.





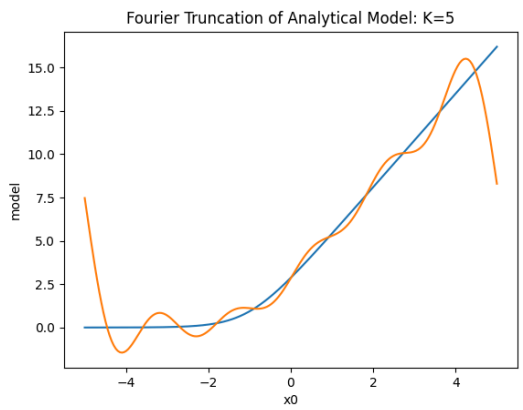
Python

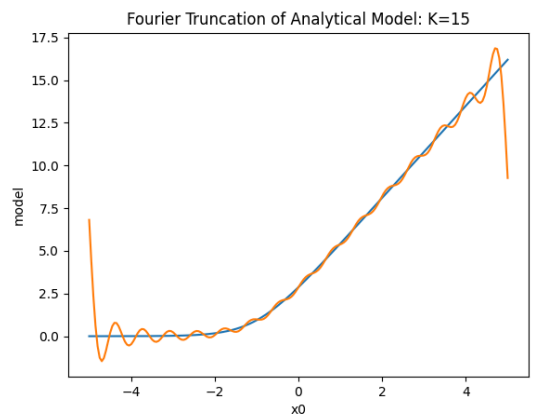
```python
T = TypeVar("T")
def kde(f: Callable[[T], float],
        sampler: Callable[[], T],
        N: int) -> Tuple[List[float], List[float]]:

    # Create N Evaluations for KDE
    evals = pseq(range(N)).map(lambda _: f(sampler())).to_list()

    # Create KDE Series
    n = len(evals)
    bw = n**(-1/5) # Scott's Rule, Naturally
    dists = (pseq(evals)
             .map(lambda xi: stats.norm(loc=xi, scale=bw))
             .to_list())
    def kde_onex(x):
        return 1/(n*bw) * (seq(dists)
                           .map(lambda k : k.pdf(x))
                           .sum())

    # Evaluate the KDE function at many points to create a smooth pdf
    fs = np.linspace(min(evals), max(evals), 1000)
    kde_values = pseq(fs).map(kde_onex).to_list()
    return fs, kde_values
```

Problem 2) Control Variates

The first order of business is finding reduced fidelity models for control variates and later multi-level monte-carlo. These reduced fidelity models must be correlated with the original model.

Analytical Model:
To create a reduced fidelity model that would have strong reason to be correlated with the original model, I took a discrete fourier transform of the analytical model, and zeroed out all terms past a user specified K. For the low fidelity control-variate model, we use K=5.



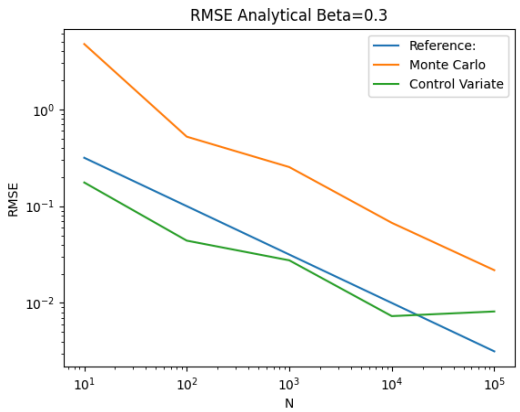


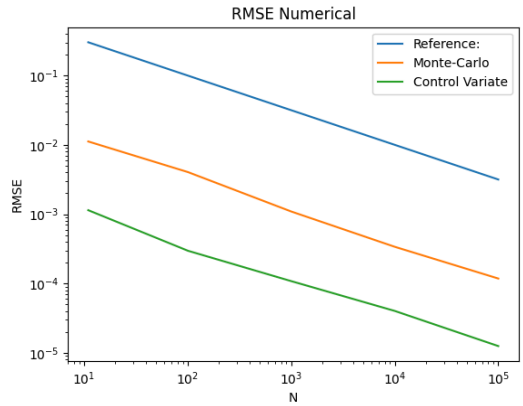
This approach provides a flexible way to generate any number of submodels of various accuracy to the original model. However, this method scales poorly with the number of points we are sampling. In particular, the process of taking the inverse Fourier Transform is $O(N^2)$. A smarter way to do this, would have been to create an analytical form of the fourier series.

Numerical Model:
To scale resolution with the numerical model, we simply change the number of grid points. The scheme should be stable given the elliptic character of the problem, so correlation shouldn't be an issue. Unlike the low-fidelity analytical function, reducing accuracy actually does decrease the simulation time.

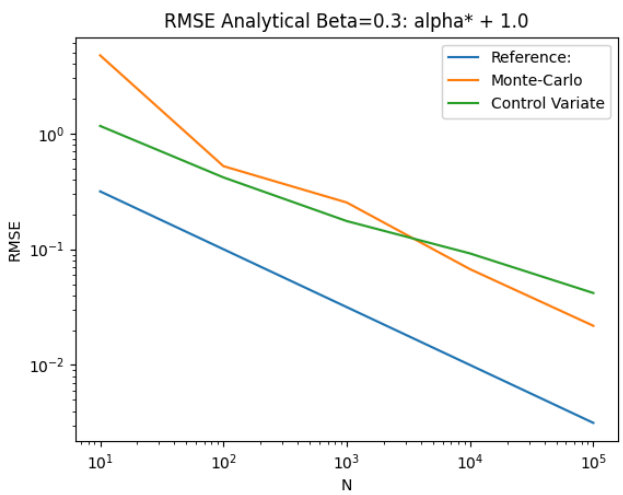
For control variates, we use 1001 x points for the high fidelity model, and 11 for the low.

* Due to its size, the control variate code can be found in the appendix





To validate that the control variate finds a reasonable alpha*, I take the average value of the 50 CV runs at 10^5, and run the control variate again, but stipulating alpha=avg_alpha* + 1.0. The following result shows, much worse convergence compared to letting the control variate algorithm compute alpha*:



Problem 3) Multi-Level Monte-Carlo

Due to time constraints, I didn't go about creating a function that turns function costs into ideal sample sizes. Not only was the determination of the costs somewhat arbitrary, but also the time to run the simulations to find suitable variances for each level would have taken a long time.

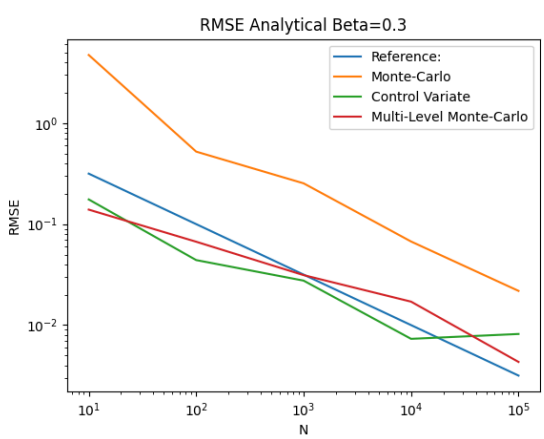
MLMC Models	Analytical Model	Numerical Model
Low Fidelity	Fourier Transform: K=15	11 Grid Points
Medium Fidelity	Fourier Transform: K=30	101 Grid Points
High Fidelity	Original Equation	1001 Grid Points

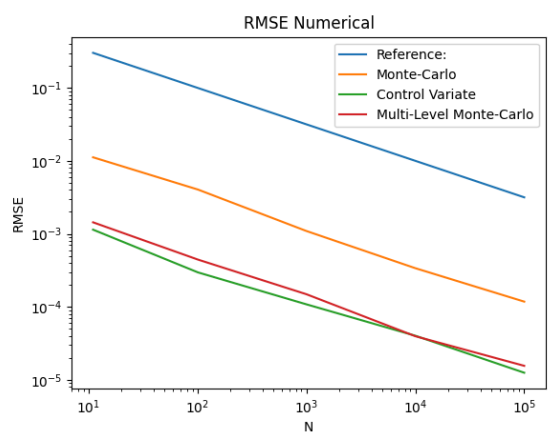
I ran each of the three following model fidelities with an increasing order of samples:

*Nvec = [N*100, N*10, N]*

Each index in *Nvec* represents the fidelity of the corresponding function, where index0 is the cheapest, lowest fidelity model, and index2 is the most expensive/most accurate model. As with all simulations, N = [10,100,1000,10000,100000]).

This cost assessment is in a way exactly the wrong cost assessment for my reduced accuracy analytical model, as the reduced order model is *more* expensive to calculate than the original function. For the numerical model however, this cost assessment is appropriate, as each drop in fidelity comes with 10 fewer grid points, and roughly 10x speedup.






```python
def multi_level(f: List[Fn[Tcv]], N: List[int], sampler) -> float:
```

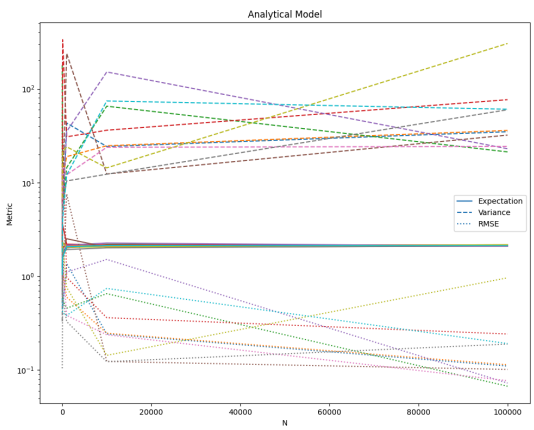
```python
def calc_evals(f,samples) -> Tuple[List[float], List[float]]:
    if f.kind == "scalar":
        evals = (pseq(samples).map(lambda x: f(x)).to_list())
    else:
        evals = f(samples)
    return evals

# Calc Exp[f0]
l0_samples = (pseq(range(N[0])).map(lambda _: sampler()).to_list())
[exp_l0,_] = monte_carlo(calc_evals(f[0], l0_samples))

# Calc Exp[flvl]
def lce(lvl: int) -> float:
    # Level Comparison Expectation E(f[lvl]-f[lvl-1])
    samples = (pseq(range(N[lvl])).map(lambda _: sampler()).to_list())
    fl_evals = calc_evals(f[lvl], samples)
    flm1_evals = calc_evals(f[lvl-1], samples)
    return 1/N[lvl] * (pseq(range(N[lvl]))
                       .map(lambda i : fl_evals[i] - flm1_evals[i])
                       .sum())

return exp_l0 + seq(range(1,(len(N)))).map(lce).sum()
```

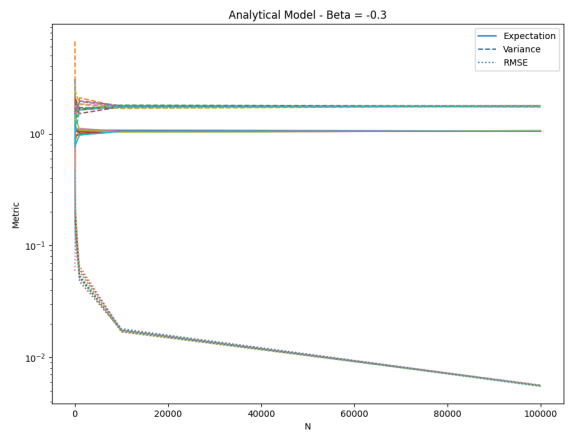
Appendix

Statistical Convergence Properties of the Models

Analytical Model

Because of the exponential term outside of the log, the analytical model can become unbound given certain distributions of x_1, and chosen values of β. Because x_1 is chosen from a non-trancated gaussian, outlier values get squared then exponentiated. This has the potential to create strong oscillations from sample to sample of the analytical model. Moreover, by finding analytical expressions of mean and variance for $e^{\beta x1^2}$ we find that the mean is bounded for β < 0.5, and the variance is bounded for β < 0.25. The following two plots show this in action for β=0.3 (left), and β=-0.3 (right):

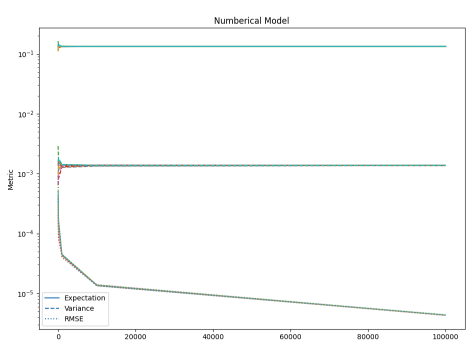




Because β < 0.5 for both cases, we can see that the mean converges to a single value as we increase N for both cases. However, for β > 0.25 (left), variance (and as a result RMSE, where RMSE = var/sqrt(N)) does not converge. For β < 0.25 (right), variance and RMSE converge.

Numerical Model

Unlike the analytical model, the numerical model converges for values of $\xi_i \in [-1,1]$:



Description of Numerical Model

The numerical model was discretized using a 2nd order central scheme, and solved implicitly via direct solve. u is evaluated at node locations, while alpha is evaluated at half points. Because the system is tri-diagonal, LU decomposition is used to speed up the direct solve.

```python
def numerical_model(xi: Tuple[float,float], N: int) -> Tuple[List[float],
List[float]]:
    [xi1,xi2] = xi
    # 0. Construct Discretization
    L = 1 # Length of domain
    dx = L/N
    def i2x(idx: int) -> float: # Index to x coordinate
        return float(idx)*dx + 0.5*dx

    # 1. Construct A Matrix (Sparse)
    def alpha(x: float) -> float:
        return 1.0 + 0.3*xi1*sin(pi*x) + 0.2*xi2*cos(2*pi*x)

    main = (seq(range(N))
             .map(lambda i: alpha(i2x(i)+0.5*dx) + alpha(i2x(i)-0.5*dx))
             .to_list())
    sub   = (seq(range(N-1))
             .map(lambda i: i+1)
             .map(lambda i: -1* alpha(i2x(i)+0.5*dx))
             .to_list())
    upper = (seq(range(N-1))
             .map(lambda i: -1 * alpha(i2x(i)-0.5*dx))
             .to_list())
    A = diags(
        diagonals=[sub, main, upper],
        offsets=[-1, 0, 1],
        format="csr"
    )

    # 2. Construct dx^2 matrix
    rhs = np.full(N, dx**2)

    # 3. Solve sytem
    xs = list(map(i2x ,range(N)))
    u = spsolve(A,rhs)

    return xs, u
```

Samplers

There are two samplers in the code, sampler_A for the analytical model, and sampler_B for the numerical model. Each observing the different distributions of the random variables in each model:

```python
# So that different parallel threads will have iid samples.
BASE_SEED = int(__import__("os").environ.get("MC_BASE_SEED", "20250227"))
SAMPLER_RNG = {"pid": None, "rng": None}

def sampler_A() -> Tuple[float, float, float]:

    import os
    import numpy as np

    pid = os.getpid()
    if SAMPLER_RNG["pid"] != pid or SAMPLER_RNG["rng"] is None:
        seed_seq = np.random.SeedSequence([BASE_SEED, pid])
        SAMPLER_RNG["pid"] = pid
        SAMPLER_RNG["rng"] = np.random.default_rng(seed_seq)

    draws = SAMPLER_RNG["rng"].normal(loc=0.0, scale=1.0, size=3)
    return float(draws[0]), float(draws[1]), float(draws[2])

def sampler_B() -> Tuple[float, float]:
    import os
    import numpy as np

    pid = os.getpid()
    if SAMPLER_RNG["pid"] != pid or SAMPLER_RNG["rng"] is None:
        seed_seq = np.random.SeedSequence([BASE_SEED, pid])
        SAMPLER_RNG["pid"] = pid
        SAMPLER_RNG["rng"] = np.random.default_rng(seed_seq)

    draws = SAMPLER_RNG["rng"].uniform(low=-1.0, high=1.0, size=3)
    return float(draws[0]), float(draws[1])
```

Control Variate Implementation

```python
Tcv = TypeVar("Tcv") # Tcv -> Type Control Variate
def control_variate(lf: Fn[Tcv], hf: Fn[Tcv],
                    N_low: int, N_high: int,
                    sampler: Callable[[], Tcv]) -> Tuple[float, float, float]:
    # 1. Take N_low samples to build expectation of hl
    def calc_evals(f,samples) -> Tuple[List[float], List[float]]:
        if f.kind == "scalar":
            evals = (pseq(samples).map(lambda x: f(x)).to_list())
        else: #kind == 'vector'
            evals = f(samples)
        return evals

    lf_samples = (pseq(range(N_low)).map(lambda _: sampler()).to_list())
    evals = calc_evals(lf, lf_samples)
    [ftild_bar, _] = monte_carlo(evals)

    # 2. Calculate lf and hf on N_high
    hf_samples = (pseq(range(N_high)).map(lambda _: sampler()).to_list())
    hf_evals = calc_evals(hf, hf_samples)
    lf_evals = calc_evals(lf, hf_samples)
    [hf_exp, hf_var] = monte_carlo(hf_evals)
    [lf_exp, lf_var] = monte_carlo(lf_evals)

    # 3. Calc Covariance
    cov = (1/N_high) * (pseq(range(N_high))
                        .map(lambda i : (hf_evals[i] - hf_exp) * (lf_evals[i] -
lf_exp))
                        .sum())
    # 4. Calc sigma_hat
    sigma_hat = (1/N_high) * (pseq(range(N_high))
                              .map(lambda i: (lf_evals[i] - lf_exp)**2)
                              .sum())

    # 5. Calc Alpha
    alpha = cov/sigma_hat

    # 6. Calc Gbar
    gbar = ((1/N_high)
            * (pseq(range(N_high))
               .map(lambda i: hf_evals[i] - alpha*lf_evals[i])
               .sum())
```

```
        + (alpha * ftild_bar))

    # 7. Calc Gvar
    gvar = hf_var - 2*alpha*cov + alpha**2 * lf_var
    return [gbar, gvar, alpha]
```